Logistic Regression

A Practical Guide to Binary Classification in Machine Learning

---

1. Introduction

Logistic Regression is a **supervised machine learning algorithm** primarily used for classification problems.

Unlike Linear Regression, which predicts continuous numerical values, Logistic Regression predicts the **probability of an observation belonging to a particular class**.

For example:

- Spam / Not Spam
- Pass / Fail
- Churn / No Churn
- Disease / No Disease
- Survived / Did Not Survive

The output probability lies between **0 and 1**, which can then be converted into a class using a decision threshold.

---

2. Why is it called Logistic Regression?

Although its name contains "Regression", Logistic Regression is mainly used for **classification**.

It uses a mathematical function called the **Sigmoid Function** to convert a linear output into a probability.

The model first calculates:

$$z = b_0 + b_1x_1 + b_2x_2 + ... + b_nx_n$$

Then applies the sigmoid function:

$$P(y = 1) = 1 / (1 + e^{-z})$$

The resulting value is always between **0 and 1**.

---

3. Sigmoid Function

The sigmoid function is the heart of Logistic Regression.

```
σ(z) = 1 / (1 + e⁻ᶻ)
```

Its output behaves approximately like this:

z	Sigmoid Output
-5	0.007
-2	0.119
0	0.500
2	0.881
5	0.993

This allows the model to interpret its output as a probability.

For example:

```
Probability = 0.82
```

The model is estimating an **82% probability** of the positive class.

---

4. Decision Boundary

After calculating the probability, we need to convert it into a class.

The commonly used threshold is:

```
Threshold = 0.50
```

Therefore:

```
If probability >= 0.50 → Class 1

If probability < 0.50 → Class 0
```

Example:

```
Prediction probability = 0.76

0.76 >= 0.50

Prediction = 1
```

The threshold does not always have to be 0.50. In real-world applications, it can be adjusted depending on whether **precision or recall** is more important.

---

5. How Logistic Regression Works

The overall process is:

```
Input Features
      ↓
Linear Equation
      ↓
Sigmoid Function
      ↓
Probability
      ↓
Decision Threshold
      ↓
Predicted Class
```

Suppose we want to predict whether a customer will leave a company.

Features could include:

```
Age
Monthly Charges
Tenure
```

```
Number of Complaints
Contract Type
```

The model calculates a score using these features and converts that score into a probability.

Example:

```
Customer → Probability of Churn = 0.87

0.87 >= 0.50

Prediction → Churn
```

---

6. Linear Equation

The model calculates a weighted combination of features.

$$z = b_0 + b_1x_1 + b_2x_2 + ... + b_nx_n$$

Where:

- b_0 = intercept
- $b_1, b_2, ..., b_n$ = model coefficients
- $x_1, x_2, ..., x_n$ = input features

The value of z is then passed through the sigmoid function.

---

7. Probability Interpretation

Logistic Regression provides more information than simply predicting a class.

For example:

```
Customer A → 0.91
Customer B → 0.63
Customer C → 0.18
```

Using a threshold of 0.50:

```
Customer A → Class 1
Customer B → Class 1
Customer C → Class 0
```

The probabilities can also help businesses identify **high-risk or high-confidence cases**.

---

8. Cost Function — Log Loss

Logistic Regression commonly uses **Log Loss**, also called **Binary Cross-Entropy Loss**.

```
Log Loss =
-[y log(p) + (1-y) log(1-p)]
```

Where:

- `y` = actual class
- `p` = predicted probability

Log Loss heavily penalizes predictions that are both:

1. Wrong
2. Highly confident

For example, predicting:

```
Actual = 1
Predicted probability = 0.99
```

is excellent.

But:

```
Actual = 1
Predicted probability = 0.01
```

is a very bad prediction and receives a large loss.

---

9. Training Logistic Regression

The model learns its coefficients during training.

A common optimization technique is **Gradient Descent**.

The general update rule is:

```
New Parameter =
Old Parameter - Learning Rate × Gradient
```

The objective is to minimize the loss function.

The learning process can be represented as:

```
Initialize Parameters
        ↓
Make Predictions
        ↓
Calculate Loss
        ↓
Calculate Gradients
        ↓
Update Parameters
        ↓
Repeat
```

---

10. Data Preprocessing

Before training a Logistic Regression model, the dataset should be properly prepared.

Common preprocessing steps

1. Handle missing values
2. Remove duplicate records
3. Correct data types
4. Handle inconsistent categories
5. Detect and handle outliers
6. Encode categorical variables
7. Scale numerical features when appropriate
8. Split data into training and testing sets

9. Check class imbalance
10. Avoid data leakage

---

11. Encoding Categorical Variables

Machine learning algorithms generally require numerical input.

For example:

```
Sex

Male
Female
```

Can be converted to:

```
Male   → 1
Female → 0
```

For categorical variables with multiple categories, techniques such as **One-Hot Encoding** are commonly used.

Example:

```
Embarked

C
Q
S
```

could become:

```
Embarked_C
Embarked_Q
Embarked_S
```

---

12. Feature Scaling

Scaling is often useful for Logistic Regression, particularly when regularization is being used.

One common technique is **Standardization**:

```
z = (x - μ) / σ
```

where:

- `μ` = mean
- `σ` = standard deviation

In Python:

```
from sklearn.preprocessing import StandardScaler

scaler = StandardScaler()

X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)
```

Important:

Fit the scaler only on the training data.

Then use the same fitted scaler to transform the test data.

---

13. Train-Test Split

The dataset should generally be divided into training and testing sets.

Example:

```
from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.2,
    random_state=42,
```

```
    stratify=y
)
```

Here:

```
80% → Training
20% → Testing
```

`random_state=42` makes the split reproducible.

`stratify=y` helps preserve the target-class distribution between the training and testing sets.

---

14. Implementing Logistic Regression in Python

```python
import pandas as pd

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import (
    accuracy_score,
    classification_report,
    confusion_matrix
)

# Load dataset
df = pd.read_csv("data.csv")

# Features and target
X = df.drop("target", axis=1)
y = df["target"]

# Train-test split
X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.2,
    random_state=42,
    stratify=y
)

# Feature scaling
scaler = StandardScaler()
```

```
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)

# Create model
model = LogisticRegression(max_iter=1000)

# Train model
model.fit(X_train, y_train)

# Predictions
y_pred = model.predict(X_test)

# Probabilities
y_prob = model.predict_proba(X_test)[:, 1]

# Evaluation
print("Accuracy:", accuracy_score(y_test, y_pred))

print("\nConfusion Matrix:")
print(confusion_matrix(y_test, y_pred))

print("\nClassification Report:")
print(classification_report(y_test, y_pred))
```

---

15. Model Evaluation

Accuracy alone is not always enough.

Important classification metrics include:

Accuracy

Measures the percentage of total predictions that are correct.

```
Accuracy =
Correct Predictions / Total Predictions
```

Precision

Precision answers:

> Of all the observations predicted as positive, how many were actually positive?

```
Precision = TP / (TP + FP)
```

High precision means fewer **False Positives**.

Recall

Recall answers:

> Of all the actual positive observations, how many did the model correctly identify?

```
Recall = TP / (TP + FN)
```

High recall means fewer **False Negatives**.

F1 Score

F1 Score combines Precision and Recall.

```
F1 = 2 × (Precision × Recall)
     / (Precision + Recall)
```

It is particularly useful when you want a balance between precision and recall.

---

16. Confusion Matrix

A confusion matrix summarizes classification predictions.

	Predicted 0	Predicted 1
Actual 0	True Negative	False Positive
Actual 1	False Negative	True Positive

True Positive — TP

Actual = 1
Predicted = 1

True Negative — TN

Actual = 0
Predicted = 0

False Positive — FP

Actual = 0
Predicted = 1

False Negative — FN

Actual = 1
Predicted = 0

---

17. Classification Report

In Python:

```
from sklearn.metrics import classification_report

print(classification_report(y_test, y_pred))
```

The report normally includes:

```
Precision
Recall
F1-score
Support
```

This provides a more detailed understanding of model performance than accuracy alone.

---

18. ROC Curve and AUC

The **ROC Curve** evaluates the model across different classification thresholds.

It plots:

```
True Positive Rate
        vs
False Positive Rate
```

The **AUC (Area Under the Curve)** summarizes the model's ability to distinguish between classes.

Generally:

```
AUC ≈ 0.5 → Poor discrimination
AUC → 1.0 → Excellent discrimination
```

ROC-AUC is especially useful when comparing classification models across different thresholds.

---

19. Regularization

Regularization helps prevent overfitting by penalizing excessively large coefficients.

Two commonly discussed types are:

L1 Regularization

Also known as **Lasso**.

It can drive some coefficients toward zero, which can help with feature selection.

L2 Regularization

Also known as **Ridge**.

It generally shrinks coefficients toward zero without necessarily making them exactly zero.

In scikit-learn:

```
model = LogisticRegression(
    penalty="l2",
    C=1.0,
    max_iter=1000
)
```

---

20. Understanding C

In scikit-learn's Logistic Regression:

```
C = inverse of regularization strength
```

Therefore:

```
Smaller C → stronger regularization

Larger C → weaker regularization
```

Example:

```
LogisticRegression(C=0.1)
```

uses stronger regularization than:

```
LogisticRegression(C=10)
```

---

21. Logistic Regression Coefficients

One advantage of Logistic Regression is that its coefficients can provide useful interpretability.

In Python:

```
print(model.coef_)
```

A positive coefficient generally indicates that increasing the corresponding feature is associated with **higher log-odds of the positive class**, while a negative coefficient indicates lower log-odds, assuming other variables are held constant.

For a more direct interpretation, coefficients can be exponentiated to obtain **odds ratios**:

```
Odds Ratio = e^(coefficient)
```

---

22. Advantages

1. Simple

Logistic Regression is relatively easy to understand and implement.

2. Fast

It can train quickly, especially on datasets that are not extremely large.

3. Interpretable

Model coefficients can provide useful insight into feature relationships.

4. Probability Output

It provides predicted probabilities rather than only class labels.

5. Strong Baseline

It is an excellent baseline model before trying more complex algorithms.

6. Regularization

L1 and L2 regularization can help control overfitting.

---

23. Limitations

Logistic Regression also has limitations.

- It may struggle with highly nonlinear relationships.
- Strong multicollinearity can affect coefficient interpretation.
- Severe class imbalance can make accuracy misleading.
- Categorical variables need suitable encoding.
- Feature engineering may be necessary for complex relationships.
- It may perform worse than nonlinear models on highly complex datasets.

---

24. When Should You Use Logistic Regression?

Logistic Regression is a good choice when:

- The target is categorical.

- You need a simple baseline.
- Interpretability is important.
- You need probability estimates.
- The relationship between features and the log-odds is reasonably suitable.
- You want a fast and computationally efficient model.

---

25. Real-World Applications

Logistic Regression is widely applicable to problems such as:

Customer Churn

Predict whether a customer will leave.

Fraud Detection

Estimate whether a transaction is potentially fraudulent.

Medical Classification

Predict whether a patient belongs to a particular risk category.

Email Spam Detection

Classify emails as spam or legitimate.

Credit Risk

Estimate the probability of loan default.

Marketing

Predict whether a customer is likely to respond to a campaign.

---

26. Complete Machine Learning Workflow

A practical Logistic Regression project can follow this workflow:

```
1. Define Business Problem
          ↓
2. Collect Dataset
          ↓
```

```
3. Understand Dataset
          ↓
4. Perform EDA
          ↓
5. Handle Missing Values
          ↓
6. Remove Duplicates
          ↓
7. Fix Data Types
          ↓
8. Handle Outliers
          ↓
9. Encode Categorical Variables
          ↓
10. Feature Engineering
          ↓
11. Feature Selection
          ↓
12. Train-Test Split
          ↓
13. Feature Scaling
          ↓
14. Train Logistic Regression
          ↓
15. Make Predictions
          ↓
16. Evaluate Model
          ↓
17. Tune Hyperparameters
          ↓
18. Select Appropriate Threshold
          ↓
19. Interpret Results
          ↓
20. Deploy / Document Model
```

---

27. Common Mistakes

Mistake 1: Scaling Before Train-Test Split

Incorrect:

```
X_scaled = scaler.fit_transform(X)

X_train, X_test, y_train, y_test = train_test_split(...)
```

This can cause data leakage.

Better:

```
X_train, X_test, y_train, y_test = train_test_split(...)

X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)
```

Mistake 2: Using Accuracy Alone

If the dataset is highly imbalanced, accuracy can give a misleading impression.

Always consider:

```
Precision
Recall
F1 Score
ROC-AUC
Confusion Matrix
```

Mistake 3: Ignoring Class Imbalance

If:

```
95% → Class 0
5%  → Class 1
```

a model predicting Class 0 almost every time could achieve high accuracy while being useless for identifying Class 1.

Mistake 4: Data Leakage

Information from the test set should not influence training.

Keep the test set separate until evaluation.

---

28. Logistic Regression vs Linear Regression

Feature	Linear Regression	Logistic Regression
Main task	Regression	Classification
Output	Continuous value	Probability
Typical output range	Any real number	0 to 1
Main function	Linear equation	Sigmoid + linear equation
Example	House price	Spam detection
Common loss	MSE	Log Loss

---

29. Key Formulas

Linear Score

$$z = b_0 + b_1x_1 + ... + b_nx_n$$

Sigmoid

$$\sigma(z) = 1 / (1 + e^{-z})$$

Precision

$$Precision = TP / (TP + FP)$$

Recall

$$Recall = TP / (TP + FN)$$

F1 Score

```
F1 = 2 × Precision × Recall
     ------------------------
       Precision + Recall
```

Standardization

```
z = (x - μ) / σ
```

Odds Ratio

```
Odds Ratio = e^(coefficient)
```

---

30. Key Takeaways

Logistic Regression is one of the **fundamental algorithms in Machine Learning** and an important model to understand before moving toward more advanced classification algorithms.

The most important concepts to remember are:

```
Logistic Regression
        ↓
Linear Combination of Features
        ↓
Sigmoid Function
        ↓
Probability
        ↓
Decision Threshold
        ↓
Class Prediction
```

You should be comfortable with:

- Sigmoid function
- Probability prediction
- Decision threshold
- Log Loss
- Gradient Descent

- Train-Test Split
- Feature Scaling
- Encoding
- Confusion Matrix
- Precision
- Recall
- F1 Score
- ROC-AUC
- Regularization
- Model coefficients
- Class imbalance

---

31. GitHub Project Structure

A professional Logistic Regression repository can look like:

```
logistic-regression/
│
├── data/
│   └── dataset.csv
│
├── notebooks/
│   └── logistic_regression.ipynb
│
├── images/
│   ├── confusion_matrix.png
│   ├── roc_curve.png
│   └── feature_coefficients.png
│
├── README.md
├── requirements.txt
└── Logistic_Regression.pdf
```

---

32. Suggested Portfolio Project

To make Logistic Regression stronger as a GitHub project, don't upload only the algorithm.

Build an end-to-end project such as:

Customer Churn Prediction

Objective

Predict whether a customer is likely to churn based on customer information.

Include:

```
EDA
↓
Data Cleaning
↓
Missing Value Analysis
↓
Categorical Encoding
↓
Feature Scaling
↓
Train-Test Split
↓
Logistic Regression
↓
Confusion Matrix
↓
Precision / Recall / F1
↓
ROC-AUC
↓
Feature Interpretation
↓
Business Recommendations
```

This demonstrates that you understand **Machine Learning as a complete workflow**, rather than simply knowing how to call:

```
LogisticRegression()
```

Conclusion

Logistic Regression is a simple yet powerful classification algorithm. Its combination of mathematical simplicity, probability-based predictions, interpretability, and computational efficiency makes it an important model for both learning and real-world applications.

For a strong Machine Learning portfolio, focus not only on training the model but also on **EDA, preprocessing, feature engineering, evaluation, interpretation, and business conclusions**.

> **Learn the algorithm. Understand the mathematics. Build the project. Explain the results.**